

- 对 ajax 的理解
- 如何创建一个 ajax 请求 / 实现一个 ajax
- 假如有多个请求，我需要通过某种顺序一次执行，有什么办法呢？
 - 方法一：Promise + async/await
 - 方法二：Jquery Deferred
- ajax axios fetch 的区别
- Axios 如何在底层实现 CSRF 防御机制
- 将原生的 ajax 封装成 promise
- ajax 返回的状态
- websocket 和 ajax 的区别是什么，websocket 的应用场景有哪些
 - 区别
 - WebSocket 的底层流程
 - WebSocket 的典型应用场景
 - AJAX 的典型应用场景

对 ajax 的理解

ajax 是通过 js 的异步通信，从服务器获取 XML 文档并从中获取数据，用于更新当前网页的对应部分，而无需刷新整个网页

如何创建一个 ajax 请求 / 实现一个 ajax

创建一个 ajax 请求：

- 创建一个 XMLHttpRequest 对象，
- 调用 open 方法创建一个 HTTP 请求，传入参数为请求方法、请求地址、是否异步、用户认证信息
- 为请求添加头信息
- 监听 readystatechange 事件：
 - 通过这个实例的 readyState 属性来判断这个 ajax 请求状态，分为 0, 1, 2, 3, 4 这四种状态（0：未初始化，1：载入/正在发送请求，2：载入完成/已经接受到响应数据，3：交互/解析数据，4：接收数据完成），
 - 当状态为 4 的时候也就是接受数据完成的时候，这时候可以通过实例的 status 属性判断这个请求是否成功，status 变化为 2xx 代表返回正常，此时

可用服务器响应 (response) 中的数据更新页面

- 调用 send 方法向服务器发送请求

```
const SERVER_URL = "/server";
let xhr = new XMLHttpRequest();
// 创建 Http 请求
xhr.open("GET", url, true);
// 设置状态监听函数
xhr.onreadystatechange = function () {
  if (this.readyState !== 4) return;
  // 当请求成功时
  if (this.status === 200) {
    handle(this.response);
  } else {
    console.error(this.statusText);
  }
};
// 设置请求失败时的监听函数
xhr.onerror = function () {
  console.error(this.statusText);
};
// 设置请求头信息
xhr.responseType = "json";
xhr.setRequestHeader("Accept", "application/json");
// 发送 Http 请求
xhr.send(null);
```

假如有多个请求，我需要通过这些 ajax 请求按照某种顺序一次执行，有什么办法呢？

方法一：Promise + async/await

```
async function send(actions, arg2) {
  for (let action of actions) {
    await send_action(action, arg2);
  }
}
```

方法二：Jquery Deffered

使 ajax 请求按照队列顺序执行，递归实现：

- 用递归逐个消费队列
- 用 \$.Deferred() 控制异步完成时机
- 用 \$.when(...).done() 实现 “等待完成再继续”

```
// 按顺序执行多个ajax命令
function send(actions, arg2) {
    $
    .when(send_action(actions[0], arg2)) // 执行当前队列的第一个任务 action[0],
    send_action 返回一个 Promise, $.when() 等待这个 Promise 完成
    .done(function () {
        // 前一个ajax回调函数完毕之后判断队列长度
        if (actions.length > 1) {
            // 队列长度大于1, 则弹出第一个, 继续递归执行该队列
            actions.shift();
            send(actions, arg2);
        }
    })
    .fail(function () {
        // 失败重试
        send(actions, arg2);
    });
}

// 处理每个命令的ajax请求以及回调函数
function send_action(command, arg2) {
    var dtd = $.Deferred(); // 定义deferred对象

    $.post("url", {
        command: command,
        arg2: arg2
    }).done(function (json) {
        json = $.parseJSON(json);
        // ...
        dtd.resolve();
    }).fail(function () {
        // 请求失败
        dtd.reject();
    });
    return dtd.promise(); // 返回Deferred对象的promise, 防止在外部修改状态
}
```

ajax axios fetch 的区别

XHR (底层能力)
↓
AJAX (技术思想/模式)
↓
Promise (异步模型)
↓

fetch (现代原生API)



axios (封装库)

XMLHttpRequest (XHR) : 浏览器提供的底层 API, 用于发 HTTP 请求、处理响应

ajax:

- AJAX 是一种思想, 本质是“用 JS 在后台发请求, 不刷新页面”。可以用 XHR 实现, 也可以用 fetch / axios 实现
- 传统 Ajax 指的是 XMLHttpRequest (XHR)
- 基于原生 XHR (XMLHttpRequest) 开发
- 针对 MVC 编程, 与当前前端流行的 MVVM 不符
- 多个请求之间如果有先后关系的话, 就会出现回调地狱 (指的是在异步编程中, 由于多次嵌套回调函数而导致代码变得复杂、难以维护的情况)
- 配置和调用方式非常混乱, 而且基于事件的异步模型不友好

fetch:

- 浏览器原生的现代 HTTP API, 作用与 XHR + AJAX 一致, 但是使用起来更加友好
- 可以替代 XMLHttpRequest
- 基于 Promise 实现, 支持 async、await, 语法简洁
- fetch 只对网络请求报错, 而将 400、500 等都作为请求成功, 需要额外处理
- 不支持 abort
- 不支持超时处理
- 无法原生监测请求的进度

axios:

- axios 是一个基于 Promise 的 HTTP 客户端
- 封装了 XHR (浏览器) 或者 http 模块 (Node.js)
- 与 fetch 不同, axios 会自动转换 JSON 数据
- 支持请求拦截与响应拦截
- 超时处理
- 支持预防 CSRF
- 更统一的 API
- 提供并发请求接口

技术	是否基于 Promise
XHR	✗ (原生不支持)

技术	是否基于 Promise
fetch	✓
axios	✓

Axios 如何在底层实现 CSRF 防御机制

CSRF(cross site request forgery) 攻击：服务器无法区分正常请求和攻击请求，因为攻击者发送请求时携带了用户点击恶意链接所包含的 token 信息

Axios 里提供了预防 csrf 攻击的功能。

防止 CSRF: 就是让你的每个请求都带一个从 cookie 中拿出的 key。根据浏览器同源策略，假冒的网站是不能读取你 cookie 中的 key 的，这样，后台就可以辨别出这个请求是否是假冒的

Axios 采用 Double Submit Cookie 策略：

- 服务端在 Cookie 中写入一个 token（如 XSRF-TOKEN）
- 客户端每次请求：
 - 判断：是否同源 / 是否允许携带 Cookie
 - 如果同源 / 允许携带 cookie(`withCredentials = true`)，从 Cookie 读取 token，放入请求头（如 X-XSRF-TOKEN）
- 服务端校验：Cookie + Header 是否一致

为什么这样可以防御 CSRF：攻击者无法读取 Cookie，也就无法伪造自定义 Header，因此无法伪造合法请求

将原生的 ajax 封装成 promise

思路：使用 `new Promise` 包装异步请求，并在请求成功时调用 `resolve()`，失败时调用 `reject()`。

```
function ajax(url, method = "GET", data = null) {
  return new Promise((resolve, reject) => {
    // 1. 创建 XMLHttpRequest 对象
    const xhr = new XMLHttpRequest();

    // 2. 初始化请求
```

```
xhr.open(method, url, true);

// 3. 设置请求头（如果需要发送 JSON 数据）
if (method === "POST") {
  xhr.setRequestHeader("Content-Type", "application/json");
}

// 4. 监听请求状态变化
xhr.onreadystatechange = function () {
  if (xhr.readyState !== 4) return; // 未完成

  // 5. 处理结果
  if ((xhr.status >= 200 && xhr.status < 300) || xhr.status === 304) {
    resolve(JSON.parse(xhr.responseText)); // 成功
  } else {
    reject(new Error(xhr.statusText)); // 失败
  }
};

// 6. 发送请求
xhr.send(data ? JSON.stringify(data) : null);
});

// 使用示例
ajax("https://api.example.com/data", "GET")
  .then((res) => console.log(res))
  .catch((err) => console.error(err));
```

ajax 返回的状态

- 0：（未初始化）还没有调用 send()方法
- 1：（载入）已调用 send()方法，正在发送请求
- 2：（载入完成）send()方法执行完成，已经接收到全部响应内容
- 3：（交互）正在解析响应内容
- 4：（完成）响应内容解析完成，可以在客户端调用了

websocket 和 ajax 的区别是什么，websocket 的应用场景有哪些

WebSocket 的诞生本质上就是为了解决 HTTP 协议本身的单向性问题：请求必须由客户端向服务端发起，然后服务端进行响应。

一旦我们需要服务端主动向客户端发送消息时就麻烦了，因为此前的 TCP 连接已经释放，根本找不到客户端在哪。

区别

1. 通信模式

- **AJAX (基于 HTTP)**：单向请求-响应 (request → response)，必须由客户端发起，服务端不能主动推送
- **WebSocket**：全双工 (双向通信)，建立连接后，客户端和服务端都可以随时发消息

AJAX 是 “拉”，WebSocket 是 “推 + 拉”

2. 连接特性

- AJAX：每次请求都是一次 HTTP 连接 (短连接或复用连接)，请求结束即断开 (逻辑上)
- WebSocket：通过 HTTP Upgrade 建立连接后，变成长连接

3. 协议开销

- AJAX：每次请求都带完整 HTTP 头，开销较大
- WebSocket：建立连接后，使用轻量级帧 (frame)，开销极小

4. 实时性

- AJAX：依赖轮询，有延迟
- WebSocket：服务端主动推送，实时性高

总结：

维度	AJAX	WebSocket
通信方式	单向 (请求驱动)	双向
连接	短连接 (逻辑上)	长连接
实时性	较差 (需轮询)	高
开销	高 (HTTP 头)	低

维度	AJAX	WebSocket
服务端推送	不支持	支持
实现复杂度	低	较高

WebSocket 的底层流程

1. 客户端发起 HTTP 请求：

```
Upgrade: websocket  
Connection: Upgrade
```

2. 服务端返回：

```
101 Switching Protocols
```

3. 连接升级为 WebSocket
4. 后续通信：不再使用 HTTP，使用自定义帧协议

WebSocket 的典型应用场景

- 实时通信类：即时聊天（IM）、在线客服、协同编辑（类似文档多人编辑）
- 实时数据流：股票行情
- 在线游戏
- 实时通知系统：消息推送

AJAX 的典型应用场景

- 低频请求：表单提交、页面加载数据
- 强依赖 HTTP 语义：RESTful API、缓存（HTTP Cache）
- 简单系统